

Authenticating Users Lab (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p4-authenticating-users-lab>  <https://github.com/learn-co-curriculum/python-p4-authenticating-users-lab/issues/>

Learning Goals

- Use the session object to authenticate a user.

Key Vocab

- **Identity and Access Management (IAM)**: a subfield of software engineering that focuses on users, their attributes, their login information and the resources that they are allowed to access.
- **Authentication**: proving one's identity to an application in order to access protected information; logging in.
- **Authorization**: allowing or disallowing access to resources based on a user's attributes.
- **Session**: the time between a user logging in and logging out of a web application.
- **Cookie**: data from a web application that is stored by the browser. The application can retrieve this data during subsequent sessions.

Introduction

In this lab, we'll continue working on the blog site from the last lab and set up a basic login feature.

There is some starter code in place for a Flask API backend and a React frontend. To get set up, run:

```
$ pipenv install && pipenv shell
$ npm install --prefix client
```

```
$ cd server  
$ flask db upgrade  
$ python seed.py
```

You can work on this lab by running the tests with `pytest -x`. It will also be helpful to see what's happening during the request/response cycle by running the app in the browser. You can run the Flask server with:

```
$ python app.py
```

And you can run React in another terminal with:

```
$ npm start --prefix client
```

You don't have to make any changes to the React code to get this lab working. The React frontend has already defined a proxy in `package.json` as shown:

```
"proxy": "http://localhost:5555",
```

The proxy avoids CORS issues and allows the server to set a session cookie to store the user's login data.

Instructions

For our basic login feature, we'll need the following functionality:

- A user can log in by providing their username in a form.
- A user can log out.
- A user can remain logged in, even after refreshing the page.

We'll need to create the resources to handle each of these features. Let's get started!

NOTE: This lab uses the Flask-Restful module rather than vanilla Flask. You do not need to use it to pass the tests, but we recommend giving it a shot.

Sessions

- Generate these resources:
- **Login** is located at `/login`.
 - It has one route, `post()`.
 - `post()` gets a `username` from `request`'s JSON.
 - `post()` retrieves the user by `username` (we made these unique for you).
 - `post()` sets the session's `user_id` value to the user's `id`.
 - `post()` returns the user as JSON with a 200 status code.
- **Logout** is located at `/logout`.
 - It has one route, `delete()`.
 - `delete()` removes the `user_id` value from the session.
 - `delete()` returns no data and a 204 (No Content) status code.
- **CheckSession** is located at `/check_session`.
 - It has one route, `get()`.
 - `get()` retrieves the `user_id` value from the session.
 - If the session has a `user_id`, `get()` returns the user as JSON with a 200 status code.
 - If the session does not have a `user_id`, `get()` returns no data and a 401 (Unauthorized) status code.

Resources

- [What is Authentication? - auth0](https://auth0.com/intro-to-iam/what-is-authentication)  <https://auth0.com/intro-to-iam/what-is-authentication>
- [API - Flask: class flask.session](https://flask.palletsprojects.com/en/2.2.x/api/#flask.session)  <https://flask.palletsprojects.com/en/2.2.x/api/#flask.session>

This tool needs to be loaded in a new browser window

Load Authenticating Users Lab (CodeGrade) in a new window